

Z INTERPRETER

Documentazione dell'Architettura Interna

Flusso di esecuzione · Strutture dati · Funzioni C

1 · Panoramica del Sistema

Lo Z Interpreter è un interprete scritto in C per il linguaggio Z, un linguaggio custom con sintassi token-based racchiusa tra simboli &. Il sistema è composto da quattro fasi principali: **lettura sorgente**, **formattazione/tokenizzazione**, **analisi statica (build_state)** e **esecuzione (parse)**. Tutto lo stato dell'interprete è contenuto nella struttura globale **VM**, evitando conflitti tra istanze multiple.

2 · Flusso di Esecuzione Principale

Fase	Funzione C	Input	Output	Note
1 · Lettura file	read_code_from_file()	argv[2] (.Zinter)	char* (codice grezzo)	Rimuove spazi, tab, newline fuori stringa
2 · Formattazione	format_code()	char* codice	program[] popolato	Divide su ':' '{ '>'; ogni token → program_line
3 · Build State	build_state()	program[]	state_stack[] popolato	Mappa { } con depth; registra funzioni/if/for/while
4 · Ricerca __start	main() loop	state_stack[]	st_ip (indice IP)	Cerca '#' o '__start' nello stack (attualmente '#')
5 · Esecuzione	parse(st_ip)	IP iniziale	Effetti collaterali	Loop while; dispatch per prefisso dell'istruzione

Nota sul punto di ingresso: nella versione attuale il main cerca il primo blocco con nome '#' nello state_stack invece di '__start' (il ramo __start è commentato). Ciò significa che l'esecuzione inizia dal primo blocco di sistema trovato, non dalla funzione main del programma Z.

3 · Struttura VM — Memoria dell'Interprete

Tutta la memoria runtime è contenuta nella struct **VM**. I #define rimappano ogni campo su alias brevi per comodità di scrittura.

Campo VM	Tipo C	Alias macro	Scopo
variable[64]	decl_var[]	variable	Variabili int dichiarate con int_&i;&
fl_variable[64]	decl_fl_var[]	fl_variable	Variabili float (tipo 'l')
char_variable[64]	decl_var_char[]	char_variable	Variabili char (tipo 'c')
array[64]	decl_arr[]	array	Array int, max 64 celle
fl_array[64]	decl_fl_arr[]	fl_array	Array float

Campo VM	Tipo C	Alias macro	Scopo
char_array[64]	decl_char_arr[]	char_array	Array char/string (tipo 's')
matrix[64]	decl_matr[]	matrix	Matrici int [row][col]
fl_matrix[64]	decl_fl_matr[]	fl_matrix	Matrici float
char_matrix[64]	decl_char_matr[]	char_matrix	Matrici char
r[16]	int[]	r	Registri interi (r0..r15)
cr[16]	char[]	cr	Registri char (cr0..cr15)
program[512]	program_line[]	program	Istruzioni tokenizzate
state_stack[64]	program_state[]	state_stack	Stack blocchi {}: funzioni, if, for, while
global_ip	int	global_ip	Instruction pointer corrente
return_state	int	return_state	Numero blocchi registrati in state_stack
deb	int	deb	Flag debug (1=attivo)
place_holder	int	place_holder	Valore int temporaneo per operazioni
char_place_holder	char	char_place_holder	Valore char temporaneo
pt_place_holder	int*	pt_place_holder	Puntatore temporaneo int
pt_char_place_holder	char*	pt_char_place_holder	Puntatore temporaneo char

4 · Strutture Dati Interne

Struct C	Campi principali	Descrizione
decl_var	name[16], value(int), pointer*	Variabile intera; il puntatore punta a sé stessa
decl_fl_var	name[16], value(float), pointer*	Variabile float
decl_var_char	name[16], value(char), pointer*	Variabile char singola
decl_arr	name[16], array_int[64](int), size	Array intero, max 64 celle
decl_fl_arr	name[16], array_int[64](float), size	Array float
decl_char_arr	name[16], array_int[256](char), size	Array char / stringa
decl_matr	name[16], matrix_int[64][64], size_first, size_sec	Matrice intera
decl_fl_matr	name[16], matrix_fl_int[64][64], ...	Matrice float
decl_char_matr	name[16], matrix_str_int[64][64], ...	Matrice char
program_line	line_number(int), instruction[512]	Una riga del programma tokenizzata
program_state	nome_function[16], posizione_ritorno, codice, posizione_skip	Voce dello stack stati: mappa apertura { con chiusura }

5.1 Funzioni di Utilità e Lookup

Funzione	Firma	Ritorno	Descrizione
<code>is_var_()</code>	<code>const char *name</code>	0/1/2	0=non trovata, 1=int, 2=char o float
<code>is_arr_()</code>	<code>const char *name</code>	0/1/2/3	0=non trovata, 1=int, 2=char, 3=float
<code>is_matrix_()</code>	<code>const char *name</code>	0/1/2/3	Come <code>is_arr_</code> ma per matrici
<code>is_reg_()</code>	<code>const char *name</code>	0/1	1 se il nome matcha rN (registro intero)
<code>get_index()</code>	<code>char data_struct_name[]</code>	<code>void*</code>	Cuore del resolver: legge la sintassi <code>&tipo;[idx]&nome;&</code> e ritorna puntatore al dato. Gestisce: numero immediato (<code>&n;&</code>), char immediato (<code>&k;&</code>), registro, matrice, array (indice numerico / variabile / vuoto per stringa intera), variabile scalare.
<code>resolve()</code>	<code>char *type, char *name</code>	<code>void*</code>	Wrapper su <code>get_index()</code> : ricostruisce la stringa <code>&type;&nome;&</code> e la passa a <code>get_index()</code>
<code>starts_with()</code>	<code>const char *str, *prefix</code>	<code>int</code>	<code>strncmp</code> -based; ritorna 1 se str inizia con prefix
<code>check_if_same_type()</code>	<code>char *arg1, char *arg2</code>	<code>int</code>	Compatibilità tipi: <code>n↔i</code> , <code>c↔k</code> , <code>s↔c</code> , <code>s↔k</code> ; ritorna <code>tru/fal</code>

5.2 Funzioni di Dichiarazione

Funzione	Parametri	Descrizione
<code>declare_variable()</code>	<code>char name[], char type</code>	Aggiunge variabile a <code>variable/fl_variable/char_variable</code> in base a <code>type (i/l/c)</code> . Valore iniziale = 0.
<code>declare_array()</code>	<code>char name[], char type, int size</code>	Alloca array in <code>array/fl_array/char_array</code> . Limita size a <code>max_number_of_cell (64)</code> . Inizializza a 0.
<code>declare_matrix()</code>	<code>char name[], char type, int s1, int s2</code>	Alloca matrice. Registra <code>size_first</code> e <code>size_sec</code> .
<code>exec_int()</code>	<code>char *text</code>	Parser per <code>'int_ &i;&nome;&'</code> e <code>'int_ &i;[dim]&nome;&'</code> . Chiama <code>declare_variable</code> o <code>declare_array</code> dopo aver estratto tipo e dimensione con <code>sscanf</code> .
<code>exec_char()</code>	<code>char *text</code>	Come <code>exec_int</code> ma per <code>'char_ &c;&nome;&'</code> / <code>'char_ &s;[dim]&nome;&'</code> .

5.3 Funzioni di Scrittura (Set)

Funzione	Parametri	Descrizione
<code>set_to_variable()</code>	<code>name, type, float value, char cvalue</code>	Usa <code>get_index</code> per trovare il puntatore al dato e vi scrive <code>value (int/float)</code> o <code>cvalue (char)</code> .
<code>set_to_array()</code>	<code>name, type, index, float value, char cvalue</code>	Scrive direttamente per indice dopo aver trovato l'array per nome.
<code>set_to_matrix()</code>	<code>name, type, row, col, float value, char cvalue</code>	Scrive nella cella <code>[row][col]</code> della matrice trovata per nome.
<code>clean_memory()</code>	<code>void</code>	Azzera i contatori di variabili e array e tutti i registri <code>r[]</code> .

5.4 Funzioni di Input/Output

Funzione	Descrizione
<code>exec_print(text)</code>	Stampa un dato via printf. Gestisce: testo puro (&s;&...&), numero puro (&n;&), variabile int/float/char, array int/float con indice, char array con indice, stringa intera (&s;[]&). Ricostruisce la stringa &tipo;&nome;& e chiama <code>get_index</code> .
<code>exec_println(text)</code>	Chiama <code>exec_print</code> e poi stampa '\n'. Usa prefisso 'println_' (8 char).
<code>exec_lnprint(text)</code>	Stampa '\n' prima, poi chiama <code>exec_print</code> . Usa prefisso 'lnprint_' (8 char).
<code>exec_lnprintln(text)</code>	Stampa '\n', chiama <code>exec_print</code> , poi '\n'. Usa prefisso 'lnprintln_' (10 char).
<code>exec_scan(text)</code>	Stub non implementato. Leggerebbe da stdin su variabili/array.

5.5 Funzioni Matematiche e di Assegnazione

`exec_math()` gestisce la keyword `=` (assegnazione) e, in prospettiva, le operazioni aritmetiche. Il parser con `sscanf` estrae `type1/operand1` (destinazione) e `type2/operand2` (sorgente). La funzione **`resolve()`** trasforma la coppia (type, name) in un puntatore `void*` con cui esegue la copia. Casi speciali: numero immediato `&n;&` → variabile temporanea N; char immediato `&k;&` → variabile temporanea K; chiamata di funzione inline con `__name()`.

Caso	Sintassi sorgente Z	Logica C
<code>N → var int</code>	<code>= &i;&var;& &n;&5&</code>	<code>N=atoi(op2); int_src=&N; *int_dest=*int_src</code>
<code>var → var</code>	<code>= &i;&v1;& &i;&v2;&</code>	<code>resolve entrambi; *dest=*src</code>
<code>arr → var</code>	<code>= &i;&v;& &i;&[N]&arr;&</code>	<code>resolve sorgente con indice; *dest=*src</code>
<code>var → arr</code>	<code>= &i;&[N]&arr;& &i;&v;&</code>	<code>resolve destinazione con indice; *dest=*src</code>
<code>arr → arr</code>	<code>= &i;&[A]&a;& &i;&[B]&b;&</code>	<code>entrambi risolti con get_index</code>
<code>K → char var</code>	<code>= &c;&var;& &k;&X;&</code>	<code>K=op2[0]; char_src=&K; *char_dest=*char_src</code>
<code>func → var</code>	<code>= &i;&var;& &__fn(args)</code>	<code>exec_funarg()</code> con opcode 1; <code>*dest=ritorno</code>

5.6 Funzioni di Confronto (exec_conf)

`exec_conf()` implementa l'operatore `==` (uguaglianza). Ha la stessa struttura di `exec_math`: estrae due operandi, li risolve con `resolve()` e confronta i puntatori. Ritorna **1 (tru)** se uguali, **0 (fal)** se diversi, **-1** in caso di errore. Supporta anche la forma con chiamata di funzione: `== &i;&var;& __fn(args)` o la forma inversa.

5.7 Funzioni di Controllo del Flusso

Funzione	Stato implementazione	Comportamento attuale / previsto
<code>exec_if(text)</code>	STUB — non implementata	Legge <code>global_ip</code> ma non salta. Dovrebbe: valutare la condizione logica nel testo, saltare a <code>posizione_skip</code> se falsa, eseguire il blocco se vera.
<code>exec_else(text)</code>	STUB — non implementata	Legge <code>global_ip</code> . Dovrebbe: saltare il blocco se il precedente if era vero, eseguirlo se falso.
<code>exec_for(text)</code>	STUB — non implementata	Dovrebbe: valutare condizione, eseguire blocco, applicare incremento, ripetere.
<code>exec_while(text)</code>	STUB — non implementata	Dovrebbe: rieseguire il blocco fintanto che la condizione è vera, usando <code>posizione_ritorno</code> dallo <code>state_stack</code> .

Funzione	Stato implementazione	Comportamento attuale / previsto
<code>exec_fence(text)</code>	PARZIALE	Gestisce <code>#debug_df / #debug_dt</code> (attiva/disattiva deb). Gestisce <code>#import_ &f;&file;&</code> (apre file ma non lo importa davvero). Il ramo <code>exec_</code> è incompleto: cerca <code>__start</code> nello stack ma non salta correttamente (usa <code>==</code> invece di <code>=</code>).
<code>exec_funarg(text, name, args, opcode)</code>	STUB	Ritorna sempre <code>&test_return_value</code> (67 hardcoded). Dovrebbe: cercare la funzione nello <code>state_stack</code> , salvare il return address, saltare al corpo, raccogliere il valore restituito.
<code>build_state()</code>	IMPLEMENTATA	Scansiona <code>program[]</code> con un algoritmo depth-first; per ogni blocco { registra nome, posizione apertura e posizione chiusura (<code>posizione_skip</code>) in <code>state_stack[]</code> .
<code>parse(idx)</code>	IMPLEMENTATA (dispatch)	Loop <code>while(global_ip <= line_idx_program)</code> ; effettua dispatch per prefisso con <code>starts_with()</code> ; incrementa <code>global_ip</code> ad ogni iterazione.

6 · Tabella Dispatch in parse()

La funzione `parse()` riconosce ogni istruzione tramite il suo prefisso e chiama la funzione handler corrispondente.

Prefisso riconosciuto	Handler chiamato	Stato
<code>=</code>	<code>exec_math()</code>	✓ implementata
<code>==</code>	<code>exec_conf()</code>	✓ implementata
<code>#</code>	<code>exec_fence()</code>	■ parziale
<code>return_</code>	(nessuno)	✗ stub
<code>__</code>	(nessuno)	✗ stub — dichiarazione funzione
<code>_</code>	(nessuno)	✗ stub — chiamata funzione
<code>int_</code>	<code>exec_int()</code>	✓ implementata
<code>char_</code>	<code>exec_char()</code>	✓ implementata
<code>if_</code>	<code>exec_if()</code>	✗ stub
<code>else_</code>	<code>exec_else()</code>	✗ stub
<code>for_</code>	<code>exec_for()</code>	✗ stub
<code>while_</code>	<code>exec_while()</code>	✗ stub
<code>set_to_matrix_</code>	<code>exec_set_to()</code>	■ parziale
<code>scan_</code>	(nessuno)	✗ stub
<code>print_</code>	<code>exec_print()</code>	✓ implementata
<code>lnprint_</code>	<code>exec_lnprint()</code>	✓ implementata
<code>println_</code>	<code>exec_println()</code>	✓ implementata
<code>lnprintln_</code>	<code>exec_lnprintln()</code>	✓ implementata

7 · Bug Noti e Limitazioni

